Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Örnek soru: iki kümenin kesişim kümesi (ekrandan, 00:46:30)
  - Soru: "Verilen iki kümenin kesişim kümesini bulan algoritmanın akış diyagramını çizip C dilinde kodunu yazınız."
  - Örnek kümeler: $A = \{1, 3, 5, 2, 4\}$ (N eleman), $B = \{6, 1, 7, 2\}$ (M eleman). Bu kümeler yalnızca örnektir. Çözüm her N ve M için çalışmalıdır, örneğe göre sabit değer yazılmaz.
  - Akış diyagramı:
    * N ve M okunur. `I ← 1 || 1 || N` döngüsüyle `kume1[I]`, `I ← 1 || 1 || M` döngüsüyle `kume2[I]` okunur. Sonra `K = 1` atanır.
    * Dış döngü `I ← 1 || 1 || N`: küme 1'in her elemanı sırayla alınır.
    * İçeride önce `J = 1` atanır. Sonra `J <= M AND kume1[I] ≠ kume2[J]` olduğu sürece `J = J + 1` yapılır (while).
    * Döngüden çıkınca `J <= M` kontrol edilir. True kolunda `kesisim[K] = kume1[I]` ve `K = K + 1` yapılır. False kolu boştur.
    * Son olarak `I ← 1 || 1 || K–1` döngüsüyle `kesisim[I]` yazdırılır ve program biter.
  - İlk çözümde iç döngü de for ile yazılmıştı: küme 2'nin tüm elemanları her seferinde taranıyordu. While ile eşleşme bulunduğu anda iç döngüden çıkılır. Gereksiz karşılaştırmalar yapılmaz.
  - Alternatif: `flag` değişkeni. Başta `flag = 0` yapılır. İçeride eşitlik bir if ile kontrol edilir ve bulunduğunda `flag = 1` yapılır. Döngüden çıkınca `flag` kontrol edilir. Bu çözüm de doğrudur.
- Hangi küme dışta, hangisi içte olmalı?
  - Soru: bir kümede birkaç milyon, diğerinde birkaç eleman varsa döngülerin sırası önemli mi?
  - Örnek: küçük küme $\{1, 3, 5, 7\}$, büyük küme $\{10, 100, 102, 105, \dots\}$ ve 1, 3, 5, 7 büyük kümenin sonunda. Hoca iki durumda da $4 \times 4 = 16$ saydı; bu sayı büyük olasılıkla yalnız ilk dört elemanla yapılan karşılaştırmaları içerir. While ile tam sayım (python ile doğrulandı): büyük küme dıştayken $16+1+2+3+4 = 26$, küçük küme dıştayken $5 + 6 + 7 + 8 = 26$.
  - Sonuç: bu örnekte while kullanıldığında da sıra fark etmez, iki durumda da eşleşme bulunan noktada çıkılır. Başka girdilerde iki sıranın sayısı büyük olasılıkla her zaman eşit çıkmaz.
- Kesişim çözümünün C kodu (öğrencinin kodu üzerinden, 01:12:36 – 01:27:06)
  - Kod ekranda birlikte incelendi. Kodun düzeltilmiş hâli "Detaylı Açıklamalar" bölümündedir.
  - Diziler sabit bir boyutla tanımlanır (örneğin `int Kume1[50]`). `int Kume1[N]` yazılamaz, çünkü bu tanımda bellekte yer program çalışmadan önce ayrılır. O anda N'nin değeri bilinmez.
  - Dinamik bellek ayırma yöntemi görüldüğünde yer, çalışma sırasında ayrılacak. O zaman bu sorun ortadan kalkacak.
  - Akış diyagramında diziler 1'den başlar. C'de 0'dan başlar. Bu yüzden C'de sınırlar hep bir eksiğine kadar gider: `i <= N-1` ya da daha sade olarak `i < N`.
  - C'de `k` 0'dan başlar. Eklenen son eleman `kesisim[k-1]` olur. Bu yüzden yazdırma döngüsü `i <= k-1` (veya `i < k`) olmalıdır. Ekrandaki sınır altyazıda iki biçimde geçer: hoca `k-2` diye okur, öğrenci "küçük k-1" der (`i <= k-2` ile `i < k-1` aynıdır). Bir öğrenci

son elemanın yazdırılmadığını fark etti ve hoca hatayı kabul etti.
  - While koşulundaki her bir koşul ayrıca parantez içine yazılmalıdır: `while((j < M) && (Kume1[i] != Kume2[j]))`.
  - If, while ile aynı hizadadır, çünkü ikisi de for'un içindedir ve if while'ın içinde değildir. Girinti (indentation) iç içelik düzenini gösterir.
  - False kolunda iş yoksa `else { }` boşuna açılmaz.
  - Çıktıda sayılar yan yana bitişik çıkıyordu. `printf("%d\n", ...)` ile her sayı ayrı satıra yazılır. `%4d` gibi formatlı yazım da vardır ama dersin amacı C'nin tüm yeteneklerini değil, akış diyagramlarını koda dökmeye yetecek kadar C'yi göstermektir.
- Çok boyutlu diziler (multi-dimensional arrays) (ekrandan, 01:32:30)
  - İki boyutlu dizi (2-D array) matristir. Satır sayısı (row number) N, sütun sayısı (column number) M ise ilk göz `mtr[0][0]`, son göz `mtr[N-1][M-1]` olur.
  - `int mtr[2][3];` tanımı 2 satır ve 3 sütundan oluşan, toplam 6 elemanlı bir matristir. Gözler: `mtr[0][0]`, `mtr[0][1]`, `mtr[0][2]`, `mtr[1][0]`, `mtr[1][1]`, `mtr[1][2]`.
  - İlk köşeli parantez satır sayısını, ikinci köşeli parantez sütun sayısını verir.
  - Bellekte saklanma: matris kağıttaki gibi iki boyutlu saklanmaz. Önce birinci satırın elemanları sütun sırasıyla, sonra ikinci satırın elemanları sütun sırasıyla arka arkaya yerleşir. Sıra `mtr[0][0], mtr[0][1], mtr[0][2], mtr[1][0], mtr[1][1], mtr[1][2]` şeklindedir.
  - Köşeli parantez sayısı artırılarak 3, 4 ve daha fazla boyutlu diziler tanımlanır. Örnek: `int kup[4][5][7];` toplam $4 \times 5 \times 7 = 140$ eleman içerir.
  - 3 boyutlu diziler fiziksel ortamı (x, y, z) modellemek için zaman zaman kullanılır. 4 ve daha fazla boyut çok sık karşımıza çıkmaz.
- Örnek soru: matrisin kolon toplamları (ekrandan, 01:58:10)
  - Soru: "Verilen bir matrisin kolon değerlerini toplayıp yeni bir dizide saklayan algoritmanın akış diyagramını çiziniz."
  - Önce kağıt üzerinde bir matris çizilir ve işin nasıl yapılacağı düşünülür. Her sütun aşağı doğru toplanır ve sonuç diziye aktarılır.
  - Okuma: `I ← 1 || 1 || N` (satırlar) dışta, `J ← 1 || 1 || M` (sütunlar) içte, `matris[I][J]` okunur.
  - Toplama: dış döngü `I ← 1 || 1 || M` (sütunlar). Her sütun için `toplam = 0`. İç döngü `J ← 1 || 1 || N` (satırlar), `toplam = toplam + matris[J][I]`. İç döngüden çıkınca `DIZI[I] = toplam`.
  - Yazdırma: `I ← 1 || 1 || M` ile `DIZI[I]` yazdırılır.
  - Tahtadaki örnek matris:

$$\begin{bmatrix} 5 & 3 & 6 & 8 \\ 7 & 2 & 4 & 2 \\ 8 & 1 & 7 & 1 \\ 9 & 0 & 3 & 0 \end{bmatrix}$$

    Kolon toplamları: $29, 6, 20, 11$.
- Örnek soru: matrisin simetrik olup olmadığı (ekrandan, 02:34:50)
  - Soru: "Verilen bir matrisin simetrik olup olmadığını bulan algoritmanın akış diyagramını çiziniz."
  - Simetrik matris: köşegene göre ayna konumundaki değerler birbirine eşittir, yani her i, j için $m[i][j] = m[j][i]$. Bir tanesi bile farklıysa matris simetrik değildir. Matris $N \times N$ olmalıdır.
  - Okuma: N okunur, `I ← 1 || 1 || N`, `J ← 1 || 1 || N` ile `m[I][J]` okunur.
  - Kontrol: `flag = 1`, `I = 1`. Dış while: `I < N AND flag = 1`. İçeride `J = I + 1`. İç while: `J <= N AND flag = 1`. İçeride `m[I][J] ≠ m[J][I]` ise `flag = 0`. Sonra `J = J + 1`. İç döngüden çıkınca `I = I + 1`.
  - Sonuç: `flag = 1` ise "Simetrik", değilse "Simetrik değil" yazdırılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Sınavda verilen örnek değerler yalnızca örnektir. Çözümü örneğin boyutlarına göre değil, genel N ve M için yazın.
- Soruda analiz istenirse verilen örnek üzerinde, örnek yoksa kendi örneğinizle mutlaka analiz yapın. "Olur gibi gözüküyor" demek yetmez.

- Adım sayısı belli değilse ve bir koşul sağlandığında döngüden erken çıkmak gerekiyorsa while kullanılır. Kesişim ve simetri sorularında while, gereksiz kontrolleri önler.
- While'dan çıktıktan sonra neden çıkıldığı bilinmez: eleman bulundu mu, yoksa dizinin sonuna mı gelindi? Bunu `J <= M` (veya `flag`) kontrolüyle anlarsınız.
- Sınır değerleri çok önemlidir. Akış diyagramında diziler 1'den, C'de 0'dan başlar. Koda geçerken sınırları buna göre değiştirin. Uç değerleri mutlaka kontrol edin.
- C'de dizi boyutu değişkenle verilemez. Sabit, yeterince büyük bir boyut verin.
- `scanf`'te değişkenin önüne `&` konur: `scanf("%d", &sayi1);`. Birden fazla değer `scanf("%d %d", &sayi1, &sayi2);` şeklinde okunabilir.
- Kolon toplamı sorusunda iç döngüde satırlar, dış döngüde sütunlar dönmelidir. Döngü sınırlarının yerleri değiştirilirse çözüm doğru çalışmaz.
- `toplam` yerine doğrudan `DIZI[I] = DIZI[I] + matris[J][I]` yazılabilir. Bu durumda dizinin bütün gözleri iç içe döngülerden önce mutlaka 0'a atanmalıdır.
- Döngü değişkenlerinin adı (I, J) önemli değildir. Önemli olan hangi döngünün satırlarda, hangisinin sütunlarda ilerlediğidir.
- Simetri kontrolünde yalnızca köşegenin üstündeki gözlere bakmak yeter. Her göz, köşegenin altındaki karşılığıyla karşılaştırılmış olur.
- Simetri sorusunda if'in true kolunda "simetrik" yazdırılamaz. Bir çiftin eşit olması bir şey söylemez. Yalnızca eşit olmayan bir çift bulunduğunda karar verilebilir.

Kısa Tekrar Notları

- Kesişim: dış döngü küme 1, iç while `J <= M AND kume1[I] ≠ kume2[J]`. Çıkışta `J <= M` ise eleman kesişime eklenir.
- Kesişimde bu örnekte hangi kümenin dışta olduğu karşılaştırma sayısını değiştirmez (örnekte iki durumda da 26).
- `flag` değişkeni, while içine yazılan koşula bir alternatiftir.
- C'de `int dizi[50];` gibi sabit boyut kullanılır, `int dizi[N];` kullanılmaz.
- C'de dizi indisleri 0 ile N−1 arasındadır. `for(i=0; i<N; i++)` kalıbı kullanılır.
- `k` 0'dan başlıyorsa k eleman eklenmiştir ve son indis `k-1`'dir.
- While'daki her koşulu ayrı parantez içine alın.
- Girinti iç içe yapıyı gösterir. Boş `else` yazılmaz.
- `printf("%d\n", x);` ile her değer ayrı satıra yazılır.
- `int mtr[N][M];`: ilk göz `mtr[0][0]`, son göz `mtr[N-1][M-1]`, toplam N×M eleman.
- Matris bellekte satır satır, art arda saklanır.
- `int kup[4][5][7];` toplam 140 eleman içerir.
- Kolon toplamı: dışta sütun (`I` 1..M), içte satır (`J` 1..N), `toplam += matris[J][I]`.
- Simetri: `I` 1..N−1, `J` I+1..N. `m[I][J] ≠ m[J][I]` ise `flag = 0`.

Detaylı Açıklamalar

Ders, kesişim kümesi sorusunun akış diyagramıyla devam eder. Önce iki iç içe for ile çözüm yazılır, sonra bir öğrencinin önerisiyle iç döngü while'a çevrilir. Böylece eşleşme bulunduğunda gereksiz karşılaştırma yapılmaz. While'dan sonra `J <= M` kontrolüyle elemanın bulunup bulunmadığı anlaşılır. Aynı işin bir `flag` değişkeniyle de yapılabileceği gösterilir. Büyük ve küçük kümenin hangisinin dışta olması gerektiği sorusu bir örnekle analiz edilir ve sıranın karşılaştırma sayısını değiştirmediği gösterilir.

Aradan sonra bir öğrencinin yazdığı C kodu ekranda incelenir. Dizilerin neden sabit boyutla tanımlanması gerektiği, 0'dan başlayan indislerin sınır değerlerini nasıl değiştirdiği, while koşullarında parantez kullanımı ve girinti konuşulur. Yazdırma döngüsündeki bir sınır hatası öğrenciler tarafından yakalanır.

Ardından çok boyutlu dizilere geçilir. İki boyutlu dizinin matris olduğu, gözlerin nasıl adlandırıldığı ve matrisin bellekte satır satır saklandığı anlatılır. Köşeli parantez sayısı artırılarak daha fazla boyutlu diziler tanımlanabileceği gösterilir.

Son bölümde iki matris sorusu çözülür. Kolon toplamları sorusunda iç ve dış döngünün neyi taraması gerektiği ve indis sırasının önemi tartışılır. Simetri sorusunda yalnızca köşegenin üstündeki gözlerin kontrol edilmesinin yeterli olduğu ve erken çıkış için while kullanılması gerektiği gösterilir. Bu soruların C kodları öğrencilere bırakılır.

Kesişim kümesi: for yerine while

İlk çözümde iki iç içe for kullanılır. Küme 1'in her elemanı küme 2'nin tüm elemanlarıyla karşılaştırılır ve eşitse kesişime eklenir. Ancak bir eşleşme bulunduğunda küme 2'nin geri kalanına bakmaya gerek yoktur. Kesişim kümesi için bir elemanın ortak olduğunu bilmek yeterlidir.

Bu yüzden iç döngü while ile yazılır:

```
J = 1
while (J <= M AND kume1[I] ≠ kume2[J])
    J = J + 1
if (J <= M)
    kesisim[K] = kume1[I]
    K = K + 1
```

While iki sebepten bitebilir. Ya eşleşme bulunmuştur (`J <= M`), ya da küme 2'nin sonuna gelinmiştir (`J = M + 1`). Son elemanda da eşleşme yoksa J bir kez daha artırılır ve M'yi geçer. Bu yüzden döngüden sonra `J <= M` kontrolü, elemanın bulunup bulunmadığını gösterir.

`K` 1'den başlar ve her eklemeden sonra artırılır. Bu yüzden döngü bittiğinde K, eleman sayısından bir fazladır ve yazdırma döngüsü K–1'e kadar gider.

Döngü sırası karşılaştırma sayısını değiştirir mi?

Bir öğrenci, küçük kümeyi büyük kümenin içinde aramanın daha avantajlı olabileceğini öne sürdü. Hoca, eşleşen elemanların büyük kümenin sonunda olduğu bir örnekle iki durumu karşılaştırdı:

- Büyük küme dışta: 10, 100, 102, 105 sırayla alınır. Her biri 1, 3, 5, 7 ile karşılaştırılır. Hoca $4 \times 4 = 16$ saydı. Büyük kümenin sonundaki 1, 3, 5, 7 de sayılınca $16 + 1 + 2 + 3 + 4 = 26$ eder (python ile doğrulandı).
- Küçük küme dışta: 1, 3, 5, 7 sırayla alınır. Her biri 10, 100, 102, 105 ile karşılaştırılır. Hoca yine $4 \times 4 = 16$ saydı. Eşleşmeler 5-8. sıralarda bulunduğu için tam sayı $5 + 6 + 7 + 8 = 26$'dır (python ile doğrulandı).

Her iki durumda da eşleşme bulunduğunda çıkılır. Bu örnekte hangi kümenin dışta olduğu sonucu değiştirmez; başka girdilerde iki sıranın sayısı büyük olasılıkla her zaman eşit çıkmaz.

Dizinin boyutu neden değişkenle verilemez?

`int Kume1[50];` gibi bir tanımda bellekteki yer program çalışmaya başlamadan ayrılır. N ve M ise program çalışırken kullanıcıdan okunur. Yer ayrılırken bu değerler bilinmediği için `int Kume1[N];` yazılamaz. Çözüm, ihtiyacı karşılayacak büyüklükte sabit bir boyut vermektir. İleride görülecek dinamik bellek ayırma yönteminde yer çalışma sırasında ayrılır ve bu kısıt ortadan kalkar.

Akış diyagramından C'ye sınır değerleri

Akış diyagramında diziler 1'den M'e kadar kullanılır. C'de diziler 0'dan $M-1$'e kadar kullanılır. Bu yüzden koda geçerken:

- `J <= M` koşulu `j <= M-1` ya da `j < M` olur.
- `J ← 1` başlangıcı `j = 0` olur.
- `K = 1` başlangıcı `k = 0` olur. Bu durumda eklenen son eleman `kesisim[k-1]`'dedir. Yazdırma döngüsü `i <= k-1` veya `i < k` olmalıdır.

Ekranda incelenen kodda yazdırma döngüsünün sınırı altyazıda iki biçimde geçer: hoca `k-2` diye okur, öğrenci "küçük k-1" der (`i <= k-2` ile `i < k-1` aynıdır). Bir öğrenci, bu hâliyle son elemanın yazdırılmadığını fark etti. Hoca, uç değerlerin kontrol edilmesi gerektiğini bir kez daha vurguladı.

Matrisin bellekte saklanması

2×3'lük bir matris kağıt üzerinde iki satır ve üç sütun olarak çizilir. Bellek ise tek boyutludur. Matrisin elemanları satır satır, art arda yerleşir:

Bellek sırası	Göz
1	`mtr[0][0]`
2	`mtr[0][1]`
3	`mtr[0][2]`
4	`mtr[1][0]`
5	`mtr[1][1]`
6	`mtr[1][2]`

Örneğin matrisin satırları `5 3 2` ve `1 0 9` ise bellekte `5, 3, 2, 1, 0, 9` sırasıyla saklanır.

Kolon toplamları: neden indis `[J][I]`?

Okuma sırasında dış döngü satırlarda, iç döngü sütunlarda ilerler. Bu yüzden `matris[I][J]` okunur. Toplama sırasında ise bir sütunun tüm satırlarını toplamak gerekir. Yani iç döngüde satır değişmeli, dış döngüde sütun bir kaymalıdır.

İlk denemede toplama da `matris[I][J]` ile yazıldı. Tahtadaki örnekle izlendiğinde `[1][1]`'den sonra `[1][2]`'ye gidildiği, yani satır boyunca ilerlendiği görüldü. İndisler `matris[J][I]` olarak değiştirildi. Böylece `[1][1]`, `[2][1]`, `[3][1]`, `[4][1]` sırasıyla birinci sütun toplanır.

Toplam, iç döngüden çıkar çıkmaz diziye yazılmalıdır. Ayrı bir döngüde yazmak mümkün değildir, çünkü bir sonraki sütuna geçildiğinde `toplam` sıfırlanır ve önceki değer kaybolur. Diziye yazarken ayrı bir K sayacı gerekmez, çünkü dış döngünün `I` değeri sütun numarasıdır ve her turda bir artar.

Simetri kontrolü: hangi gözlere bakılmalı?

İlk akla gelen çözüm, tüm matrisi satır satır taramaktır. Bu çözüm yanlış değildir, ama gereksiz kontroller yapar:

- Köşegen elemanları kendileriyle karşılaştırılır.
- Her çift iki kez kontrol edilir. Örneğin önce `m[1][2]` ile `m[2][1]`, sonra `m[2][1]` ile `m[1][2]`.

Yalnızca köşegenin üstündeki gözlere bakmak yeterlidir. I. satırda J, `I + 1`'den başlar ve N'e kadar gider. Son satırda köşegenin sağında göz kalmadığı için son satıra bakılmaz. Bu yüzden dış döngü koşulu `I <= N` yerine `I < N` olur.

While kullanılır, çünkü eşit olmayan ilk çift bulunduğunda matrisin simetrik olmadığı kesinleşir ve geri kalan kontrollere gerek kalmaz. `flag = 0` yapıldığında iki döngü de biter.

Bir öğrencinin önerdiği alternatif: `flag` yerine `m[I][J] = m[J][I]` koşulu doğrudan iç while'a yazılır. While'dan çıkınca `J <= N` kontrol edilir. Hoca bunun da çalışır göründüğünü, ama kesin karar için analiz yapmak gerektiğini söyledi.

Kesişim kümesi: C kodu

Derste incelenen kodun düzeltilmiş hâli (yazdırma sınırı `k-1`, while koşulları parantezli, çıktıda `\n`). `gcc -Wall` ile derlendi ve $A = \{1, 3, 5, 2, 4\}$, $B = \{6, 1, 7, 2\}$ için `1` ve `2` çıktısını verdi.

```c
#include <stdio.h>

int main()
{
    int N,M;
    int Kume1[50],Kume2[50];
    int i,j;
    int kesisim[50];
```

```
    int k=0;

    printf("Ilk kumenin eleman sayisini girin:");
    scanf("%d",&N);
    printf("Ikinci kumenin eleman sayisini girin:");
    scanf("%d",&M);

    for(i=0;i<N;i++)
    {
        printf("1. Kumenin %d. degeri girin: ",i+1);
        scanf("%d",&Kume1[i]);
    }
    for(i=0;i<M;i++)
    {
        printf("2. Kumenin %d. degeri girin: ",i+1);
        scanf("%d",&Kume2[i]);
    }
    for(i=0;i<N;i++)
    {
        j=0;
        while((j<M) && (Kume1[i]!=Kume2[j]))
        {
            j=j+1;
        }
        if(j<M)
        {
            kesisim[k]=Kume1[i];
            k=k+1;
        }
    }
    for(i=0;i<=k-1;i++)
    {
        printf("%d\n",kesisim[i]);
    }
    return 0;
}
```

Kolon toplamları ve simetri: C kodu

Derste bu iki sorunun yalnızca akış diyagramı çizildi ve kodu öğrencilere bırakıldı. Aşağıdaki kodlar akış diyagramlarının C'ye çevrilmiş hâlidir. `gcc -Wall` ile derlendi ve tahtadaki örneklerle test edildi (kolon toplamları: 29, 6, 20, 11).

```
/* Kolon toplamları */
for(i=0;i<M;i++)          /* dış döngü: sütunlar */
{
    toplam=0;
    for(j=0;j<N;j++)      /* iç döngü: satırlar */
        toplam=toplam+matris[j][i];
    dizi[i]=toplam;
}

/* Simetri kontrolü */
flag=1;
i=0;
while((i<N-1) && (flag==1))       /* son satıra bakmaya gerek yok */
{
    j=i+1;                        /* sadece köşegenin üstü */
    while((j<N) && (flag==1))
    {
        if(m[i][j]!=m[j][i])
```

```
            flag=0;
        j=j+1;
    }
    i=i+1;
}
if(flag==1)
    printf("Simetrik\n");
else
    printf("Simetrik degil\n");
```

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.